

Trabalho 1: Threads e semáforos com conta bancária e múltiplos caixas

Sistemas Operacionais (AL0341) - Prof. Dr. Claudio Schepke

Amanda Dias de Souza (2510101045)

1. Introdução

A aplicação simula uma conta bancária cujo saldo é operado por N caixas eletrônicos que executam em paralelo, cada um em uma thread. Toda operação (depósito ou saque), segue três passos: **ler** o saldo, **calcular** o novo valor e **escrever** o resultado. Quando dois caixas leem o mesmo saldo antes que qualquer um deles escreva, a segunda escrita apaga a primeira, fenômeno conhecido como **atualização perdida**, e o saldo final deixa de corresponder à soma das operações. O programa reproduz esse erro de forma controlada e mostra que um semáforo binário em torno dos três passos o elimina.

2. Implementação

O programa foi escrito em Python 3.14 com o módulo `threading`. A classe `Conta` (`conta.py`) guarda o saldo e expõe os métodos `depositar` e `sacar`, ambos implementados como `ler`, `calcular` e `escrever`. Cada caixa é uma `threading.Thread` que percorre uma lista fixa de operações (`caixa.py`). A conta recebe, na construção, um semáforo opcional: quando ele é `None`, o acesso ao saldo é livre; quando é um `threading.Semaphore(1)`, os três passos passam a formar uma região crítica que apenas uma thread atravessa por vez. Como os dois modos compartilham o mesmo código e a mesma lista de operações, a única diferença entre eles é a presença do semáforo, o que torna a comparação direta. O código-fonte completo pode ser acessado em: <https://github.com/amandadiasdev/simulador-bancario-threads>.

Detalhes de implementação. O interpretador CPython possui o Global Interpreter Lock (GIL), que só permite trocar de thread entre instruções do interpretador. Por isso, uma atribuição como `saldo += x` raramente é interrompida entre a leitura e a escrita, e a condição de corrida, embora exista, quase nunca se manifesta. Para torná-la observável, a conta executa `time.sleep(delay)` entre a leitura e a escrita. O `sleep` não cria o erro: ele libera o GIL exatamente no ponto em que a preempção do sistema operacional agiria em uma linguagem sem GIL ou em processos separados. Com `delay = 0` a corrida já se manifesta; com `delay = 1` ms torna-se possível também medir o custo de serializar a região crítica.

Funcionalidades auxiliares. A opção `--verbose` imprime um extrato numerado de cada leitura e escrita, com uma cor por caixa, seguido de uma auditoria que identifica, pelos números dos eventos, quais escritas foram apagadas e por quais. A conta não detecta nada em tempo real; a auditoria é feita sobre o extrato, após a execução. Para que a ordem impressa corresponda à ordem real dos acessos à memória, cada registro é feito dentro de uma trava própria do extrato, que não cobre o `sleep` e, portanto, não elimina a corrida. O script `bateria.py` executa os cenários experimentais nos dois modos e grava uma linha por execução em `resultados/resultados.csv`, com cenário, parâmetros, modo, repetição, saldo esperado, saldo final, erro e tempo.

3. Resultados e Discussão

Em todos os cenários o saldo inicial é 1000 e cada caixa recebe uma lista fixa de operações de +10 ou -10, sorteada com semente 42. Como as operações são conhecidas de antemão, o saldo esperado também é, e o erro de uma execução é definido como a diferença entre o saldo esperado e o saldo final. Foram definidos seis cenários, cada um executado nos dois modos, com e sem semáforo. O cenário A é o caso base, com cinco caixas e 200 operações por caixa sem espera artificial. O cenário B repete essa configuração com espera de 1 ms entre leitura e escrita, para medir o custo da serialização, e o cenário C reduz o problema a dois caixas e 100 operações, o menor caso em que o erro ainda se manifesta. O cenário D amplia a escala para vinte caixas e 2000 operações. O cenário E repete o caso base com o modo verboso ligado e a saída descartada, para isolar o efeito da instrumentação sobre o tempo. Por fim, o cenário F serve de controle: com um único caixa não há concorrência e, portanto, não deve haver erro em nenhum dos modos. As repetições variam de 5 a 20 por cenário, totalizando 130 execuções em Python 3.14.6 sobre um computador Linux Ubuntu 26.04 LTS.

Tabela 1. Tempo de execução e taxas de erro em múltiplos cenários de teste.

Cenário	Caixas	Operações por caixa	Espera (ms)	Semáforo	Saldo incorreto	Tempo médio (s)
A	5	200	0	não	19 de 20	0,0025 ± 0,0007
	5	200	0	sim	0 de 20	0,0032 ± 0,0003
B	5	200	1	não	10 de 10	0,264 ± 0,008
	5	200	1	sim	0 de 10	1,306 ± 0,020
C	2	100	1	não	10 de 10	0,128 ± 0,001
	2	100	1	sim	0 de 10	0,254 ± 0,003
D	20	2000	0	não	5 de 5	0,135 ± 0,004
	20	2000	0	sim	0 de 5	0,248 ± 0,001
E (verboso)	5	200	0	não	10 de 10	0,0140 ± 0,0005
	5	200	0	sim	0 de 10	0,0099 ± 0,0002
F (controle)	1	1000	0	não	0 de 10	0,0006 ± 0,0000
	1	1000	0	sim	0 de 10	0,0011 ± 0,0000

4. Conclusão

Sem semáforo, o saldo final divergiu do esperado em 54 das 55 execuções concorrentes. A única execução correta foi coincidência de escalonamento, não garantia. **Com semáforo, o saldo esteve correto em todas as 65 execuções.** O cenário de controle, com um único caixa, não apresentou erro em nenhum modo, o que confirma que a causa do problema é o acesso concorrente à variável compartilhada.

Também percebe-se que o semáforo tem custo. Com espera de 1 ms, a região crítica deixa de se sobrepor entre threads e o tempo cresce cerca de cinco vezes para cinco caixas; sem espera artificial, o acréscimo fica entre 1,3 e 1,8 vez. O cenário E revela um **efeito de instrumentação**: com o extrato ligado, o modo com semáforo foi o mais rápido, porque a trava do extrato e a impressão passam a dominar o tempo e, sem semáforo, cinco threads disputam essa trava a cada evento. Medidas de desempenho devem, portanto, ser feitas sem o modo verboso.